Quarto Manuals

Executable operator manuals for reproducible scientific workflows

Tinashe M. Tapera, Michelle Audirac, Danielle Braun, Christopher Golden

HARVARD T.H. CHAN
SCHOOL OF PUBLIC HEALTH

The problem

Scientific workflows depend on local platforms, configuration, versions, permissions, and ordered state changes. A README can describe the workflow steps, but cannot easily show whether it is safe to proceed between steps or whether the workflow succeeded.

Our approach

Quarto Manuals are ordered `.qmd` pages for domain-competent scientists who run software. Each page makes inputs, choices, actions, artifacts, and checks visible. The rendered manual is a readable interface; the source page remains the operational record.

The page contract

Each operational page answers three questions: what must already be true, what change(s) will I make, and what evidence says it worked?

Prerequisite: Start From Current State

Check for visible files, tools, configuration, or prior artifacts.

```python
# Example: check that a file exists
import os
assert os.path.exists("data.csv"), "data.csv is required!"
```

Procedure: Make One Bounded Change

Perform one bounded transformation while keeping meaningful choices and execution boundaries visible.

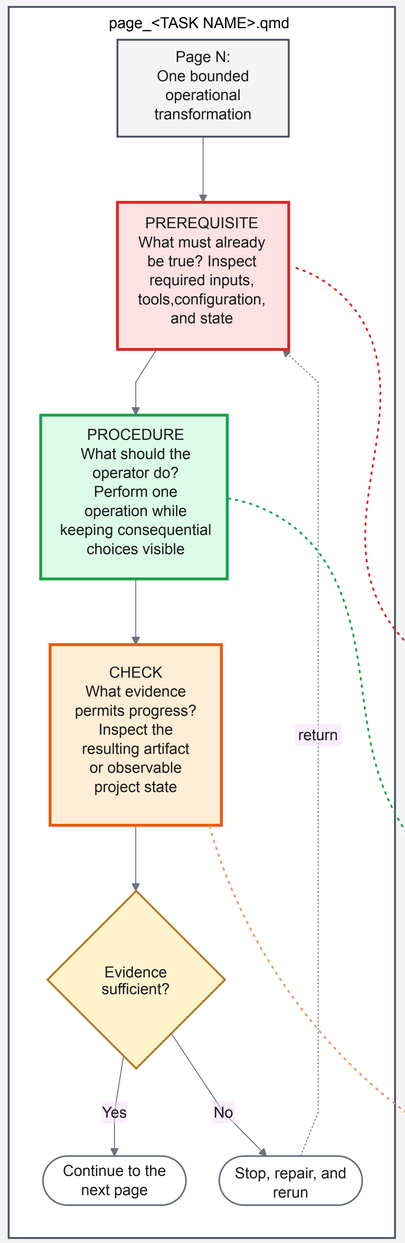
```python
# Example: manipulate the data.csv file
import pandas as pd
data = pd.read_csv("data.csv")
# Perform a bounded change, e.g., filter rows
data_filtered = data[data['value'] > 0]
# Save the result to a new file
data_filtered.to_csv("data_filtered.csv", index=False)
```

Check: Verify The Result

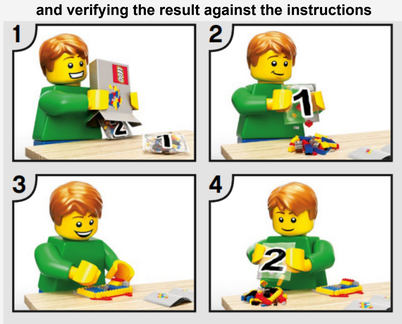
Prove that the intended state or artifact exists; a successful command alone is not evidence that the workflow succeeded.

```python
# Assert that the filtered data has the expected properties
assert not data_filtered.empty, "Filtered data is empty"
```

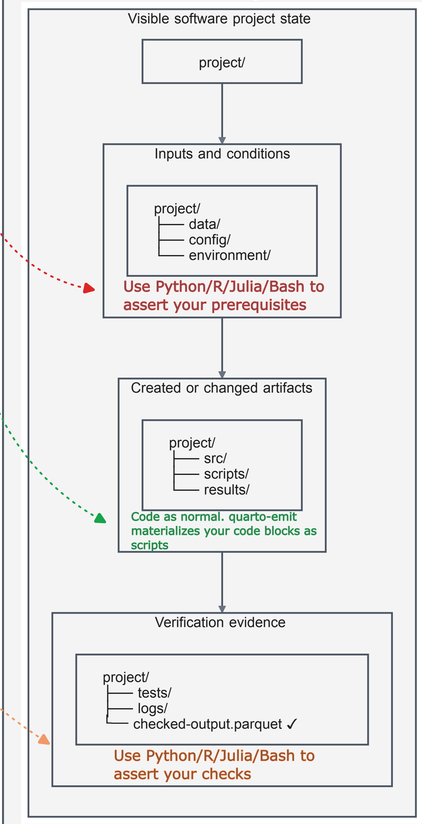
Structure of a Quarto Manual Page



Like playing with LEGO — a build is constructed by taking inventory of your pieces, putting them together, and verifying the result against the instructions



Your Assembled Output is a Reproducible Scientific Software Project



Explore Quarto Manuals

goldenplanetaryhealthlab.github.io/QuartoManual

Internals

- **Quarto:** a manual is powered by Quarto, a user-friendly scientific publishing system that uses plain text notebooks. Quarto projects can be seamlessly rendered to HTML, PDF, or other formats.
- **Book-as-Manual:** a manual is a Book project with ordered pages and navigation—mimicking a physical instruction manual.
- **Extensions:** thin extensions compose existing Quarto features. We introduce `quarto-emit`, an extension that reviews source code blocks and materializes them as code, tests, scripts, and configuration.

Why this exists

Scientific software workflows are often too complex for a README and too judgment-heavy to hide behind a single command-line tool. Thorough documentation may *describe* a workflow, but websites, tutorials, and blog posts lack the *interactivity* needed to navigate whether the current state/step is valid, what local environment adjustments are needed, or what evidence permits proceeding to the next step.

Quarto Manuals provide a reproducible handoff: the smallest reliable sequence of actions that leaves visible artifacts and confirms progress. They're designed to preserve operator authority where **local constraints** and **scientific judgment** matter, while making repeatable work **mechanical and transparent**.

Future development opportunities

A Manual page provides a transparent interface between an operator and an **agentic coding assistant**:

- **Bounded context:** prerequisites verbally identify the approved specification, relevant files, environment, and constraints.
- **Bounded change:** procedures authorize one inspectable modification; an agent may propose or emit source, tests, scripts, and configuration.
- **Verifiable handoff:** checks become executable acceptance criteria, with a human deciding whether the next page may run.

Because **Quarto Manuals promote human writing and review of scientific prose**, this structure supports agentic workflows, including:

- **Turn-based execution** — the traditional human operator and agent take turns writing expectations, executing code, and passing tests;
- **Spec-driven development** (SDD), where a human operator provides a descriptive specification and an agentic system proposes code, configuration, and tests for review;
- **Data Analysis Decision Records** (DADR), where a human operator and an agentic system record the rationale for data analysis decisions, including the specification, code, and tests.

Takeaway: Keep workflow decisions, state transitions, artifacts, and check points visible—so human-directed and AI-assisted work is transparent!